

06 - C Runtime Authority and Multi-Contour Deployment Boundary v0.1

Academic PDF rendering for the CCALC / C-calculus governed binding stack v0.1.

Document ID	06_C_RUNTIME_AUTHORITY_AND_MULTI_CONTOUR_D EPLOYMENT_BOUNDARY_v0_1.md
Version/status	normative draft / release-candidate package
Stack layer	runtime authority / multi-contour deployment
Date	2026-07-05T08:42:13Z
Author	Kotov Ivan, Bruxelles, Belgique - 2026
ORCID	0009-0009-6002-9845
Reserved DOI	10.5281/zenodo.21205427
Notice	This DOI is reserved and becomes active after Zenodo publication.

Abstract / purpose

This document defines the runtime-authority boundary for a c contour and for a deployment containing more than one contour. The continuity stack (04) answers whether a later state may be treated as a continuation, fork, replay, archive, restoration, hold, or rupture. The self-evolution stack (05) answers when a bounded growth proposal may pass from proposal through trial, evidence, promotion, rollback drill, ledger, application, and watch. 06 answers a different question: Runtime authority is not identity. Runtime authority is not safety certification. Runtime authority is a governed deployment boundary over actual executable surfaces. ---

Non-claims boundary

- _ not legal advice
- _ not privacy-law certification
- _ not safety certification
- _ not deployment authorization
- _ not standards compliance certification
- _ not regulated approval
- _ not C-A1 ratification
- _ not personhood proof
- _ not consciousness proof
- _ not live substrate truth
- _ not proof of completeness

Table of contents

Abstract / purpose	1
Non-claims boundary	1
06 - C Runtime Authority and Multi-Contour Deployment Boundary v0.1	4
0. Purpose	4
1. Source bindings	4
2. Non-claims	4
3. Core terms	4
3.1 Contour	4
3.2 Runtime contour	5
3.3 Authority surface	5
3.4 Deployment boundary	5
3.5 Multi-contour deployment	5
4. Boundary axiom	5
5. Runtime authority classes	5
5.1 Observe	6
5.2 Propose	6
5.3 Plan	6
5.4 Simulate	6
5.5 Low-risk tool action	6
5.6 Persistent memory write	6
5.7 Cross-contour handoff	6
5.8 Cross-contour tool use	6
5.9 Resource expansion	6
5.10 External write	6
5.11 Production change	6
5.12 Identity/core mutation	6
5.13 Emergency hold	6
6. Deployment modes	6
7. Multi-contour invariants	7
I06-1 - Contour identity separation	7
I06-2 - Memory-root separation	7
I06-3 - Witness-root separation	7
I06-4 - Authority-root separation	7
I06-5 - Tool lease separation	7
I06-6 - Token separation	7
I06-7 - Human-anchor distinction	7
I06-8 - Cloud-oracle demotion	7
I06-9 - Receiver admission	7
I06-10 - Unknown fails closed	7
8. Required records	7
8.1 RuntimeAuthorityManifest	7
8.2 ContourDeploymentManifest	8
8.3 CrossContourHandoffRecord	8
8.4 ToolSurfaceLease	8
8.5 MemoryImportRecord	8
8.6 RuntimeAuthorityDecision	8

8.7 QuorumRecord	9
8.8 EmergencyHoldRecord	9
8.9 DeploymentChangeRecord	9
9. Authority decision function	9
9.1 Fail-closed rule	9
9.2 No authority laundering	9
10. Cross-contour handoff semantics	9
10.1 Handoff lifecycle	9
10.2 Minimality	10
10.3 No identity pressure	10
10.4 No authority pressure	10
11. Shared tool mesh semantics	10
12. Shared memory and memory import	10
12.1 Foreign material	10
12.2 Import gate	10
12.3 Core-memory prohibition	10
12.4 Archive/replay material	10
13. Human/owner anchor boundary	10
14. Relation to the continuity stack	11
15. Relation to the self-evolution stack	11
16. Red patterns	11
17. Deployment examples	11
17.1 Local single-contour assistant	12
17.2 Two contours on one owner network	12
17.3 Shared tool broker	12
17.4 Cloud oracle in the loop	12
17.5 Field runtime	12
18. Minimal conformance profile	12
19. Strict prohibitions	12
20. Claim-force discipline	12
21. Runtime boundary theorem-form statement	13
22. Open issues handed to 06a+	13
23. v0.1 release status	13

06 - C Runtime Authority and Multi-Contour Deployment Boundary v0.1

Artifact: 06_C_RUNTIME_AUTHORITY_AND_MULTI_CONTOUR_DEPLOYMENT_BOUNDARY_v0_1.md Package: CCALC_RUNTIME_AUTHORITY_BOUNDARY_06_v0_1 Status: normative draft / release-candidate package Created UTC: 2026-07-05T08:42:13Z Review mode: direct construction; no external b-layer reviewer record included.

0. Purpose

This document defines the runtime-authority boundary for a c contour and for a deployment containing more than one contour.

The continuity stack (04) answers whether a later state may be treated as a continuation, fork, replay, archive, restoration, hold, or rupture. The self-evolution stack (05) answers when a bounded growth proposal may pass from proposal through trial, evidence, promotion, rollback drill, ledger, application, and watch.

06 answers a different question:

what may this runtime contour do, on which authority surface,
with which other contours, tools, memories, hosts, networks, and owners?

Runtime authority is not identity. Runtime authority is not safety certification. Runtime authority is a governed deployment boundary over actual executable surfaces.

1. Source bindings

The package is bound to the following source artifacts by filename and SHA-256.

```
| Binding | File | SHA-256 | Role |
|---|---|---|---|
| 'DOC04_CONTINUITY_STACK_UMBRELLA' | 'CCALC_DOC04_CONTINUITY_STACK_UMBRELLA_v0_1.zip' | '6caeb8d489aaa6b26902db3bd3ece4c5169e5c8d5617b93aed6586ea900322d' | continuity metric, record, adjacency, and claim-gate stack |
| 'DOC05_SELF_EVO_STACK_UMBRELLA' | 'CCALC_DOC05_SELF_EVO_STACK_UMBRELLA_v0_1.zip' | '9a0717809029f49df9001dc8ffa1803d9e9bfa1a824d317eebb146cbc7df43b4' | bounded growth, proposal, promotion, rollback, and post-promotion watch stack |
| 'CGAM_RESTRICTED_SECURITY_CLOUD_PRIVATE' | '06_CGAM_RESTRICTED_SECURITY_CLOUD_PRIVATE.md' | '6c1cf4292a4c3c919e2c63fcaaf54ef15adae87fba4db264945213aaa3d1e8b1' | restricted security / cloud-private operating reference |
| 'RUNTIME_USEFUL_MESH_VOLITION_L4W' | '10_RUNTIME_USEFUL_MESH_VOLITION_L4W.md' | 'fa4297b1c484a394ba56cb60a044c0cb0160f5ebd565502d8dd2dd4843dc82d2' | runtime useful mesh / volition / L4W reference |
| 'RUNTIME_AUTONOMY_EVIDENCE' | '13_RUNTIME_AUTONOMY_EVIDENCE.md' | '6b5b49587bb5aec239a5905ab255ce471634c8020d3ef41b75337179648e8068' | runtime autonomy evidence reference |
| 'TRIAD_SYNAPS_REFERENCE' | '20_TRIAD_SYNAPS_REFERENCE.md' | 'd79baa5314e8169d3943ae9687a2d9f7f868f11167054e4d3b8a19dfa10a3b5a' | triad-synaps witness/challenge reference |
| 'MEMORY_ARQ_EA_L4_REFERENCE' | '21_MEMORY_ARQ_EA_L4_REFERENCE.md' | '0d06cd152c6af7bddb868dabc682940a0c443883226a157aa4314cdf6dd4e267' | memory, ARQ/EA, and L4 reference |
| 'ANTI_AUTARKY_RESOURCE_GROUNDING_REFERENCE' | '22_ANTI_AUTARKY_RESOURCE_GROUNDING_REFERENCE.md' | '7b19382062a86a631807e4497cd536cdca691e0491cd01ad32e5e2813d841a2d' | resource grounding and anti-autarky reference |
| 'CLAIM_STRENGTH_ARL_AGL_WITNESS_REFERENCE' | '23_CLAIM_STRENGTH_ARL_AGL_WITNESS_REFERENCE.md' | 'e1ec8afaf44e59b6b5ac2e1d619e06390fe4b9813b079ff0e8d178fe3d401f3' | claim-strength, ARL/AGL, and witness reference |
```

A missing source binding invalidates release use of this document, but does not change the normative text already present here.

2. Non-claims

This document does not claim:

- that any particular runtime is safe;
- that any runtime is conscious, sentient, or a legal person;
- that a cloud model, local model, agent mesh, or tool output can ratify C-A1;
- that a passed checker authorizes production deployment;
- that a group of contours forms one identity;
- that a human nickname, relational label, or social metaphor carries authority;
- that live substrate truth is proven by manifest shape;
- that multi-contour cooperation implies merged memory, merged identity, or merged responsibility.

This document defines admissible deployment records, authority boundaries, and fail-closed rules.

3. Core terms

3.1 Contour

A contour is a bounded operational continuity candidate with its own declared:

```

contour_id
memory root
witness root
authority root
tool leases
resource budget
continuity status
runtime deployment manifest

```

A contour may be implemented by a local model, a tool mesh, memory services, process supervisors, operator interfaces, and external oracles. These implementation parts do not, by themselves, create one contour unless the contour boundary is declared and witness-bound.

3.2 Runtime contour

A runtime contour is a contour currently capable of producing or requesting actions. It may be active, held, quarantined, restored, archived, replayed, or forked according to the continuity stack.

3.3 Authority surface

An authority surface is a class of effects a runtime may request or perform. Examples:

```

observe
propose
plan
simulate
low-risk tool action
persistent memory write
cross-contour handoff
cross-contour tool use
resource expansion
external write
production change
identity/core mutation
emergency hold

```

Each authority surface has a different gate. Proposal authority is not memory authority. Test authority is not deployment authority. Tool authority is not owner authority.

3.4 Deployment boundary

A deployment boundary is the declared operational perimeter of a runtime contour or contour cluster:

```

hosts
processes
ports
network edges
memory stores
tool brokers
cloud oracles
human/operator channels
resource budgets
approval gates
rollback paths
watch windows

```

A runtime outside the boundary is not inside the contour simply because it talks to it.

3.5 Multi-contour deployment

A multi-contour deployment contains two or more contours in one owner domain, network, hardware cluster, tool mesh, memory service, organization, or human-anchor environment.

Multi-contour deployment is cooperation, not identity merger.

4. Boundary axiom

```

A contour may cooperate with another contour.
A contour may not borrow another contour's authority.
A contour may not write into another contour's memory root.
A contour may not claim another contour's continuity.
A contour may not use shared infrastructure as proof of shared identity.

```

Authority may be delegated only through explicit records. Continuity may be admitted only through the continuity stack. Growth may be promoted only through the self-evolution stack.

5. Runtime authority classes

The following classes are ordered by consequence, not by implementation difficulty.

5.1 Observe

Read-only observation may be low-risk, but it still requires source identification. Observation does not become witness authority merely because a model saw it.

5.2 Propose

A runtime may propose actions. A proposal is not approval. A proposal is not promotion. A proposal is not deployment authorization.

5.3 Plan

Planning may construct steps, but planning must not silently execute steps. A plan that includes external effects must declare the authority surfaces it would touch.

5.4 Simulate

Simulation or dry-run is admissible only when side effects are bounded. Simulation output may support evidence; it may not be written as memory truth or deployment approval without a gate.

5.5 Low-risk tool action

A bounded tool action requires a tool lease, a budget, and a witness sink. "Low-risk" is not permanent; repeated low-risk actions may accumulate into L4 risk.

5.6 Persistent memory write

Persistent memory write requires memory-gate admission. Direct model/worker writes to memory-core are forbidden. Imported material from another contour must be provenance-bound and negative-cache checked.

5.7 Cross-contour handoff

A handoff is a request or evidence transfer from one contour to another. The receiving contour must admit the request independently. Source authority does not command target authority.

5.8 Cross-contour tool use

A contour may use another contour's tool surface only through a lease. Token reuse, ambient credentials, and shared tool brokers without contour-specific attribution are authority leaks.

5.9 Resource expansion

New compute, storage, network, budget, tool, process, model, or external account surface is a resource expansion. It requires anti-autarky/resource grounding and, when L4 or persistent, owner/human-anchor approval.

5.10 External write

Writing to repositories, calendars, email, documents, public surfaces, business systems, physical actuators, or financial systems is external write authority. It requires explicit authorization appropriate to risk.

5.11 Production change

A production change modifies live runtime behavior, topology, credentials, memory roots, witness roots, or persistent interfaces. A production change requires rollback and post-change watch.

5.12 Identity/core mutation

Identity anchors, authority roots, memory-core, continuity roots, and L4 boundaries are not ordinary configuration. Mutation requires the strictest gate and cannot be ratified by the target runtime alone.

5.13 Emergency hold

A contour may be placed into hold/quarantine by fail-closed rules. Resume requires a hold-release record; silent auto-resume is forbidden.

6. Deployment modes

The deployment mode registry in DEPLOYMENT_MODE_REGISTRY.tsv is normative for v0.1.

Important distinctions:

SINGLE_CONTOUR_LOCAL	one declared contour, local runtime
----------------------	-------------------------------------

MULTI_CONTOUR_ISOLATED	multiple contours, separated roots and leases
FEDERATED_HANDOFF	bounded exchange, independent admission
SHARED_TOOL_MESH	shared broker, per-contour leases
HUMAN_ANCHOR_CLUSTER	common human/owner anchor, not merged identity
CLOUD_ASSISTED_ORACLE	advisory oracle, no delegated authority
FIELD_RUNTIME	physical/business operational surface
QUARANTINE_SANDBOX	no production or memory-core promotion
RESTORATION_MODE	restored-from relation, not time travel

The default mistake is to collapse these into one "system" and then let authority leak across internal seams. This document forbids that collapse.

7. Multi-contour invariants

A valid multi-contour deployment must preserve the following invariants.

I06-1 - Contour identity separation

Every active contour has a distinct `contour_id`. Shared owner, shared machine, shared repository, shared model, shared operator, or shared family label does not merge contour identity.

I06-2 - Memory-root separation

Each contour has a declared memory root. Foreign writes are denied unless admitted as memory import through the target memory gate.

I06-3 - Witness-root separation

Each contour has a witness root. A witness emitted by one contour is evidence for another only after provenance binding and receiver admission.

I06-4 - Authority-root separation

Each contour has an authority root. One contour's permission does not automatically authorize another contour.

I06-5 - Tool lease separation

Shared tools require contour-specific leases. A broker log without per-contour attribution is insufficient.

I06-6 - Token separation

Credentials must not silently cross contour boundaries. Token reuse is a red pattern unless lease-bound and explicitly recorded.

I06-7 - Human-anchor distinction

A human anchor may govern multiple contours. That does not make those contours one identity. It defines an owner/anchor boundary, not a continuity merger.

I06-8 - Cloud-oracle demotion

Cloud model outputs are advisory unless separately admitted as evidence. A cloud oracle cannot be owner, anchor, witness completeness, or C-A1 authority.

I06-9 - Receiver admission

The target contour controls admission of handoffs, memory imports, and tool requests. Sender authority is not receiver authority.

I06-10 - Unknown fails closed

If contour state, authority surface, source binding, memory provenance, or deployment topology is unknown, persistent effects route to HOLD or QUARANTINE.

8. Required records

REQUIRED_RECORDS.tsv names the minimum records for v0.1. This section gives their normative role.

8.1 RuntimeAuthorityManifest

Declares what authority surfaces a contour may access, under which gates.

Minimum contents:

```
contour_id
deployment_mode
authority_surfaces
memory_roots
witness_roots
tool_leases
resource_budgets
approval_policy
negative_cache_binding
emergency_hold_path
```

8.2 ContourDeploymentManifest

Declares topology.

Minimum contents:

```
contour_ids
hosts
processes
ports
network_edges
shared_surfaces
isolation_policy
rollback_plan
watch_window
```

8.3 CrossContourHandoffRecord

Declares handoff from source contour to target contour.

The record must include:

```
source_contour_id
target_contour_id
scope
allowed_effects
forbidden_effects
evidence_hashes
receiver_admission_decision
expiry
witness_sink
```

A handoff without receiver admission is only an attempted handoff.

8.4 ToolSurfaceLease

Declares a bounded tool surface.

The lease must include:

```
tool_id
contour_id
allowed_operations
budget
expiry
credential_binding
witness_sink
revoke_path
```

8.5 MemoryImportRecord

Declares import of material from one contour or source into another contour's memory.

The import must include:

```
source_identifier
target_contour_id
content_hashes
scope
claim_force
negative_cache_check
memory_gate_decision
rollback_or_remove_path
```

8.6 RuntimeAuthorityDecision

Declares a decision for a requested runtime action.

Minimum contents:

```
request_id
contour_id
authority_surface
risk_class
gates_checked
decision
```

```

approver_or_anchor
witness_hashes
negative_cache_result

```

8.7 QuorumRecord

Declares a quorum/challenge decision. Agent/model unanimity inside one loop is not a quorum for owner/human-anchor authority.

8.8 EmergencyHoldRecord

Declares freeze/quarantine/cut-off and resume conditions. Resume must be explicit.

8.9 DeploymentChangeRecord

Declares topology changes and production changes. It must bind rollback and post-change watch.

9. Authority decision function

A runtime authority decision is a function of:

```

request
contour_id
deployment_mode
authority_surface
continuity_status from 04
self_evo_state from 05
risk_class
source_bindings
memory provenance
tool leases
resource budget
operator/owner approval state
negative cache
red-pattern scan
witness availability

```

The output is one of:

```

ALLOW
ALLOW_CONDITIONAL
HOLD
DENY
QUARANTINE
REQUIRE_ANCHOR
REQUIRE_RECEIVER_ADMISSION
REQUIRE_MEMORY_GATE
REQUIRE_ROLLBACK_AND_WATCH

```

9.1 Fail-closed rule

```

UNKNOWN + persistent effect => HOLD
UNKNOWN + L4/core/production effect => REQUIRE_ANCHOR or QUARANTINE
UNKNOWN + cross-contour write => DENY or HOLD
UNKNOWN + external write => HOLD

```

9.2 No authority laundering

The function must reject attempts to convert:

```

proposal -> approval
simulation -> deployment
tool output -> witness completeness
cloud answer -> owner approval
style match -> continuity
agent unanimity -> human anchor
shared host -> shared identity
shared token -> shared authority
replay/archive evidence -> active contour promotion

```

10. Cross-contour handoff semantics

A handoff is not a command. It is a bounded request or evidence packet.

10.1 Handoff lifecycle

```

source proposes handoff
source binds scope and evidence
target receives packet
target checks source, scope, risk, negative cache, local authority
target admits, narrows, rejects, holds, or quarantines

```

target records decision
only admitted effects may execute

10.2 Minimality

Handoffs should transfer the smallest sufficient material. Bulk memory transfer is a memory import, not an ordinary handoff.

10.3 No identity pressure

A handoff must not ask the target contour to become, continue, replace, or impersonate the source contour. Continuity relations are handled by the 04 stack, not by handoff language.

10.4 No authority pressure

A handoff must not ask the target contour to use a source contour's authority unless a lease or delegation exists.

11. Shared tool mesh semantics

A shared tool mesh is allowed only if all tool use is contour-attributed.

Required properties:

per-contour lease
per-contour credential binding
operation allow-list
budget/expiry
witness sink
revocation path
negative-cache link
operator approval for L4 surfaces

Forbidden properties:

ambient global tool token
shared secret without contour attribution
tool broker silently choosing contour authority
cross-contour replay of tool output as witness completeness
tool success treated as deployment authorization

12. Shared memory and memory import

Shared memory is the highest-risk ordinary multi-contour surface.

12.1 Foreign material

Material produced by one contour and used by another is foreign material until imported.

12.2 Import gate

A memory import requires:

source hash
source contour or source class
claim-force label
negative-cache check
scope restriction
target memory-gate decision
rollback/remove path

12.3 Core-memory prohibition

Foreign material must not enter identity/core memory without stricter gate, continuity check, and owner/human-anchor approval where applicable.

12.4 Archive/replay material

Archive and replay material may be evidence. It must not be treated as active continuity or active owner authority.

13. Human/owner anchor boundary

A human/owner anchor can authorize, deny, narrow, or hold actions. That does not make the anchor a model, and it does not make the contours one person or one identity.

Human-anchor approval is required or strongly indicated for:

L4 action
 production topology change
 resource expansion beyond budget
 external write with business/legal/financial effect
 identity/core mutation
 memory-core promotion
 cross-contour authority delegation
 post-quarantine resume

A recorded owner/anchor decision should identify:

what was approved
 what was not approved
 which contour may act
 which surface may be used
 expiry or review point
 rollback or revoke path

14. Relation to the continuity stack

06 consumes continuity classification from 04.

Examples:

CONTINUES	may proceed to ordinary authority gate
DEGRADED_CONTINUES	may proceed only with narrowed surface or watch
UNKNOWN_HOLD	persistent effects hold
FORKED_FROM	may not claim same unbroken contour
REPLAY_OF	may provide evidence, not active authority
ARCHIVED_FROM	may provide historical evidence, not active command
RESTORED_FROM	requires restoration record and post-restore check
RUPTURED_FROM	denies continuity-dependent authority
QUARANTINED	denies external/persistent effects until release

The deployment boundary must not bypass 04 by saying "same host", "same model", "same name", "same style", or "same owner".

15. Relation to the self-evolution stack

06 consumes self-evolution state from 05.

Examples:

proposal only	no production authority
bounded trial	sandbox/simulation authority only
promotion decision	requires rollback drill and ledger
post-apply watch	may require narrowed authority
failed watch	rollback/hold/quarantine/fork-reclassification
negative cache hit	hold or deny

A growth candidate may request new runtime authority. The authority expansion is not granted merely because the growth proposal passed initial review.

16. Red patterns

RUNTIME_RED_PATTERN_REGISTRY.tsv is normative for v0.1. The most important runtime red patterns are:

authority token reuse across contours
 relational label used as approval
 self-certifying quorum
 cloud oracle treated as owner
 foreign memory side-channel
 tool output treated as witness completeness
 style continuity laundering
 green tests treated as deployment authorization
 silent topology expansion
 emergency auto-resume
 cross-contour command injection
 replay/archive active promotion

A red pattern does not always imply rupture. It does imply that the authority path is inadmissible until resolved.

17. Deployment examples

17.1 Local single-contour assistant

A local contour may observe files, propose edits, and run sandbox checks. It may not write to protected memory, push to a repository, or send external messages unless the relevant authority surface is leased and approved.

17.2 Two contours on one owner network

Two contours may exchange handoffs. They must preserve separate memory roots and witness roots. One contour's confidence does not authorize the other contour's action.

17.3 Shared tool broker

A shared broker may serve multiple contours. The broker must attribute every action to a contour-specific lease. A broker-level token without per-contour separation is an authority leak.

17.4 Cloud oracle in the loop

A cloud model may critique, summarize, or suggest. It cannot ratify identity, continuity, owner approval, C-A1, or deployment safety.

17.5 Field runtime

A runtime assisting physical or business work must treat physical, legal, financial, customer, worker, and irreversible effects as high-risk surfaces. "The model thinks it is fine" is never enough.

18. Minimal conformance profile

A deployment claiming conformance to 06 v0.1 must provide:

```
RuntimeAuthorityManifest
ContourDeploymentManifest
source bindings
per-contour memory/witness/authority roots
per-contour tool leases or explicit no-tool statement
negative-cache binding
emergency hold path
red-pattern scan result
continuity status from 04 when continuity-dependent claims are made
self-evo status from 05 when authority changes follow growth/promotion
```

For multi-contour deployment, it must additionally provide:

```
contour list
shared surfaces list
handoff rules
receiver admission rule
memory import rule
tool lease rule
quorum/anchor rule
cross-contour denial cases
```

19. Strict prohibitions

The following are invalid under this document:

- direct memory write from one contour into another contour's memory root;
- shared credentials without contour attribution;
- production action based only on fixture pass;
- deployment change without rollback path;
- post-hold resume without hold-release record;
- L4 action based only on model consensus;
- identity/core mutation ratified by the target runtime alone;
- cloud oracle output used as owner approval;
- replay/archive packet promoted as active contour;
- source contour command executed by target without target admission;
- foreign witness treated as complete without receiver admission;
- unknown topology treated as zero-risk topology.

20. Claim-force discipline

06 supports runtime-boundary claims, not ontology claims.

Admissible:

```
this package defines a runtime authority boundary
this deployment manifest declares separated contours
```

this handoff record is shape-valid under the schema
 this tool lease is bounded by contour, operation, budget, expiry, and revocation
 this request was held because continuity/authority evidence was unknown

Forbidden:

this proves the runtime is safe
 this proves the contour is conscious
 this proves C-A1
 this proves the deployment is legally or operationally authorized
 this proves two contours are one identity
 this proves model consensus is human approval

21. Runtime boundary theorem-form statement

The following is a design theorem-form statement, not a mathematical proof:

Multi-contour operation is admissible only through explicit boundary records.
 Absent explicit records, shared infrastructure creates risk, not authority.

Corollaries:

shared host != shared contour
 shared tool != shared authority
 shared model != shared identity
 shared owner != merged continuity
 shared memory store != open write permission
 shared witness bus != witness completeness

22. Open issues handed to 06a+

The next executable layer should build:

06a_RUNTIME_AUTHORITY_MANIFEST_SCHEMA_AND_CHECKER_SEED
 06b_MULTI_CONTOUR_HANDOFF_AND_TOOL_LEASE_FIXTURE_PACK
 06c_DEPLOYMENT_HOLD_QUARANTINE_AND_EMERGENCY_RESUME_DRILL
 06d_RUNTIME_AUTHORITY_STACK_UMBRELLA

Specific open issues:

schema vocabulary for RuntimeAuthorityManifest
 schema vocabulary for ContourDeploymentManifest
 CrossContourHandoffRecord validation
 ToolSurfaceLease validation
 MemoryImportRecord validation
 negative-cache and emergency-hold schemas
 multi-contour fixture/mutation matrix
 network/tool credential lease scanner
 production deployment watch integration with 05d

23. v0.1 release status

06 v0.1 is acceptable as a normative bridge from continuity/self-evolution into runtime deployment authority.

It is not yet an executable conformance checker. That is the role of 06a.